Selenium with Python -- Full Notes

1. What Is Selenium?

Selenium is an open-source tool used to automate web browsers. It allows Python programs to interact with websites like a human user.

Selenium can perform actions such as:

- Opening a website.
- Clicking buttons and links.
- Entering text into input fields.
- Submitting forms.
- Selecting dropdown options.
- Taking screenshots.
- Handling alerts and popups.
- Switching between browser windows and tabs.
- Automating JavaScript-heavy websites.
- Testing web applications.

Example

javascript.snippet JAVASCRIPT

```
from selenium import webdriverdriver = webdriver.Chrome()driver.get("https://www.python
.
org")print(driver.title)driver.quit()
```

---

2. Why Do We Use Selenium?

Selenium is useful when a website requires browser interaction.

Common use cases:

1. Web application testing.
2. Browser automation.
3. Form automation.
4. End-to-end testing.
5. Testing JavaScript-based websites.
6. Automating repetitive browser tasks.
7. Extracting information from dynamic web pages when permitted.

Important: Selenium is not primarily a web scraping library. For static HTML scraping, `requests` and Beautiful Soup are often simpler and faster.

---

3. Selenium Components

Selenium has several important components.

3.1 Selenium WebDriver

WebDriver allows a program to control a browser.

Examples:

- Chrome
- Firefox
- Edge
- Safari

3.2 Selenium IDE

A browser extension used to record and replay browser interactions.

3.3 Selenium Grid

Allows automated tests to run across multiple machines, browsers, and environments.

3.4 Selenium Manager

Modern Selenium releases include Selenium Manager, which can automatically discover, download, and manage compatible browser drivers in many common setups.

---

4. Install Selenium

Install Selenium using pip:

```javascript.snippet JAVASCRIPT
pip install selenium
```

Check the installed version:

```javascript.snippet
python -c "import selenium; print(selenium.__version__)"
```

Requirements

- Python installed.
- Selenium installed.
- A supported browser installed, such as Google Chrome or Firefox.

Modern Selenium can often manage the browser driver automatically.

---

5. Import Selenium

```javascript.snippet
from selenium import webdriverfrom selenium.webdriver.common.by import Byfrom selenium.
webdriver
.common.keys import Keysfrom selenium.webdriver.support.ui import WebDriverWaitfrom
selenium
.webdriver.support import expected_conditions as ECfrom selenium.webdriver.support.ui
import
 Selectfrom selenium.common.exceptions import TimeoutException
```

You do not need to import every module in every program. Import only what your program uses.

---

6. Open and Close a Browser

Open Chrome

```javascript.snippet
from selenium import webdriverdriver = webdriver.Chrome()
```

Open a Website

```javascript.snippet
driver.get("https://www.google.com")
```

Get Page Title

```javascript.snippet JAVASCRIPT
print(driver.title)
```

Get Current URL

```javascript.snippet JAVASCRIPT
print(driver.current_url)
```

Get Page Source

```javascript.snippet JAVASCRIPT
print(driver.page_source)
```

Close the Current Window

```javascript.snippet JAVASCRIPT
driver.close()
```

Quit the Entire Browser Session

```javascript.snippet JAVASCRIPT
driver.quit()
```

Difference:

- `close()` closes the current browser window.
- `quit()` closes all windows associated with the WebDriver session and ends that session.

Prefer `quit()` when your automation task is complete.

---

7. Basic Selenium Program

```javascript.snippet JAVASCRIPT
from selenium import webdriverdriver = webdriver.Chrome()try:    driver.get("https://
www
.python.org")    print("Title:", driver.title)    print("URL:", driver.current_url)
finally
:    driver.quit()
```

The `finally` block helps ensure the browser session is closed even if an error occurs.

---

8. Selenium Locators

A **locator** identifies an HTML element on a web page.

Selenium supports several locator strategies through `By`.

8.1 ID

HTML:

Python:

```javascript.snippet JAVASCRIPT
element = driver.find_element(By.ID, "username")
```

8.2 NAME

HTML:

Python:

```javascript.snippet JAVASCRIPT
element = driver.find_element(By.NAME, "email")
```

8.3 CLASS_NAME

HTML:

```javascript.snippet JAVASCRIPT
Login
```

Python:

```
javascript.snippet JAVASCRIPT

element = driver.find_element(By.CLASS_NAME, "login-button")
```

Use a single class name, not a space-separated list of classes.

8.4 TAG_NAME

```
javascript.snippet JAVASCRIPT

element = driver.find_element(By.TAG_NAME, "input")
```

8.5 LINK_TEXT

HTML:

```
javascript.snippet JAVASCRIPT

About Us
```

Python:

```
javascript.snippet JAVASCRIPT

element = driver.find_element(By.LINK_TEXT, "About Us")
```

8.6 PARTIAL_LINK_TEXT

```
javascript.snippet JAVASCRIPT

element = driver.find_element(    By.PARTIAL_LINK_TEXT,    "About")
```

8.7 CSS_SELECTOR

HTML:

Python:

```
javascript.snippet JAVASCRIPT

element = driver.find_element(    By.CSS_SELECTOR,    "#email")
```

Other CSS examples:

```javascript.snippet JAVASCRIPT
driver.find_element(By.CSS_SELECTOR, ".form-control")driver.find_element(By.
CSS_SELECTOR
, "input[name='email']")driver.find_element(By.CSS_SELECTOR, "button.login-button")
driver
.find_element(By.CSS_SELECTOR, "div.container p")
```

8.8 XPATH

XPath can locate elements using HTML structure, attributes, or text.

```javascript.snippet JAVASCRIPT
driver.find_element(By.XPATH, "//input[@id='email']")
```

Other examples:

```javascript.snippet JAVASCRIPT
driver.find_element(By.XPATH, "//button")driver.find_element(By.XPATH,
"//a[@href='/about']"
)driver.find_element(By.XPATH, "//button[text()='Login']")driver.find_element(By.XPATH,

"//input[@name='username']")
```

Locator Comparison

Locator	Example
ID	By.ID, "email"
NAME	By.NAME, "username"
CLASS_NAME	By.CLASS_NAME, "btn"
TAG_NAME	By.TAG_NAME, "input"
LINK_TEXT	By.LINK_TEXT, "About"
PARTIAL_LINK_TEXT	By.PARTIAL_LINK_TEXT, "Abo"
CSS_SELECTOR	By.CSS_SELECTOR, "#email"
XPATH	By.XPATH, "//input[@id='email']"

Best practice: Prefer stable IDs or well-designed CSS selectors when available. Use XPath when its structural or text-based capabilities are helpful.

9. find_element() vs find_elements()

find_element()

Returns the first matching element.

```javascript.snippet JAVASCRIPT
element = driver.find_element(By.TAG_NAME, "p")
```

If no element matches, Selenium raises `NoSuchElementException`.

find_elements()

Returns a list of all matching elements.

```javascript.snippet JAVASCRIPT
elements = driver.find_elements(By.TAG_NAME, "p")for element in elements:    print(
element
.text)
```

If no elements match, it returns an empty list.

Example

```javascript.snippet JAVASCRIPT
from selenium import webdriverfrom selenium.webdriver.common.by import Bydriver =
webdriver
.Chrome()try:    driver.get("https://example.com
")    heading = driver.find_element(By.TAG_NAME, "
h1")    print(heading.text)    paragraphs = driver.find_elements(By.TAG_NAME, "p")
for
 paragraph in paragraphs:        print(paragraph.text)finally:    driver.quit()
```

10. Interacting with Web Elements

10.1 Click an Element

```javascript.snippet
button = driver.find_element(By.ID, "submit")button.click()
```

10.2 Enter Text

```javascript.snippet
username = driver.find_element(By.ID, "username")username.send_keys("my_username")
```

10.3 Clear an Input

```javascript.snippet
username.clear()
```

10.4 Select All and Replace Text

```javascript.snippet
username.clear()username.send_keys("new_username")
```

10.5 Press Keyboard Keys

```javascript.snippet
search = driver.find_element(By.NAME, "q")search.send_keys("Python Selenium")search.
send_keys
(Keys.ENTER)
```

Other useful keys:

```javascript.snippet
Keys.ENTERKeys.TABKeys.ESCAPEKeys.BACKSPACEKeys.DELETEKeys.ARROW_DOWNKeys.ARROW_UPKeys.
CONTROLKeys
.SHIFT
```

For combinations:

```javascript.snippet
search.send_keys(Keys.CONTROL, "a")
```

Keyboard shortcuts can differ across operating systems.

11. Getting Element Information

Get Visible Text

```javascript
print(element.text)
```

Get an Attribute

```javascript
print(element.get_attribute("href"))print(element.get_attribute("class
"))print(element.get_attribute("
value"))
```

Get a DOM Attribute

```javascript
print(element.get_dom_attribute("href"))
```

`get_attribute()` can return a current property value for certain attributes, while `get_dom_attribute()` retrieves the attribute value from the DOM.

Check Whether an Element Is Displayed

```javascript
print(element.is_displayed())
```

Check Whether an Element Is Enabled

```javascript
print(element.is_enabled())
```

Check Whether a Checkbox Is Selected

```javascript
print(element.is_selected())
```

12. Waiting in Selenium

Web pages often need time to load. Waiting is essential for reliable automation.

12.1 Time.sleep()

```javascript.snippet JAVASCRIPT
import timetime.sleep(3)
```

This pauses execution for three seconds.

Limitation: It always waits for the full duration, even if the element becomes ready earlier.

12.2 Implicit Wait

```javascript.snippet JAVASCRIPT
driver.implicitly_wait(10)
```

Selenium waits up to ten seconds when searching for an element that is not immediately available.

Implicit waits apply to element location operations.

12.3 Explicit Wait

Explicit waits wait for a particular condition.

```javascript.snippet JAVASCRIPT
wait = WebDriverWait(driver, 10)element = wait.until(    EC.
visibility_of_element_located
(        (By.ID, "username")    ))
```

This waits up to ten seconds for the element to become visible.

Common Expected Conditions

```
javascript.snippet                                                    JAVASCRIPT

EC.presence_of_element_located(locator)EC.visibility_of_element_located(locator)EC.
element_to_be_clickable
(locator)EC.invisibility_of_element_located(locator)EC.title_contains("Python
")EC.url_contains("
dashboard")EC.alert_is_present()
```

Example: Wait for a Button

```
javascript.snippet                                                    JAVASCRIPT

from selenium import webdriverfrom selenium.webdriver.common.by import Byfrom selenium.
webdriver
.support.ui import WebDriverWaitfrom selenium.webdriver.support import
expected_conditions
 as ECdriver = webdriver.Chrome()try:    driver.get("https://example.com
")    wait = WebDriverWait(driver, 10)    button = wait.until(        EC.element_to_be_clickable(
submit")        )    )    button.click()finally:    driver.quit()
```

Best practice: Prefer explicit waits for dynamic pages. Avoid mixing implicit and explicit waits because their combined timing can be unpredictable.

---

13. Handling Dropdowns

For a standard HTML element, use Selenium's `Select` class.

HTML:

```
javascript.snippet                                                    JAVASCRIPT

    Bangladesh    United States    United Kingdom
```

Python:

```
javascript.snippet                                                    JAVASCRIPT

from selenium.webdriver.support.ui import Selectdropdown = Select(    driver.
find_element
(By.ID, "country"))dropdown.select_by_visible_text("Bangladesh")
```

Select by Value

```javascript.snippet
dropdown.select_by_value("bd")
```

Select by Index

```javascript.snippet
dropdown.select_by_index(0)
```

Get All Options

```javascript.snippet
for option in dropdown.options:    print(option.text)
```

Get Selected Option

```javascript.snippet
print(dropdown.first_selected_option.text)
```

Note: `Select` works with standard HTML elements, not custom dropdowns built with or other elements. Custom dropdowns require ordinary clicking and locating techniques.

---

14. Handling Checkboxes and Radio Buttons

Checkbox

```javascript.snippet
checkbox = driver.find_element(By.ID, "agree")if not checkbox.is_selected():
checkbox
.click()
```

Radio Button

```javascript.snippet
radio = driver.find_element(By.ID, "male")if not radio.is_selected():    radio.click()
```

Check the current state before clicking when you want a specific final state.

---

15. Handling Alerts and JavaScript Popups

Selenium can interact with JavaScript alerts, confirmations, and prompts.

Accept an Alert

```javascript
alert = driver.switch_to.alertalert.accept()
```

Dismiss an Alert

```javascript
alert = driver.switch_to.alertalert.dismiss()
```

Read Alert Text

```javascript
print(alert.text)
```

Enter Text into a Prompt

```javascript
alert.send_keys("Hello")alert.accept()
```

Wait for an Alert

```javascript
wait = WebDriverWait(driver, 10)alert = wait.until(    EC.alert_is_present())alert.
accept
()
```

16. Handling Frames and Iframes

An iframe contains another document inside a web page. Switch to it before locating its internal elements.

Switch Using an ID or Name

```javascript.snippet JAVASCRIPT
driver.switch_to.frame("payment-frame")
```

Switch Using a Web Element

```javascript.snippet JAVASCRIPT
iframe = driver.find_element(    By.TAG_NAME, "iframe")driver.switch_to.frame(iframe)
```

Switch Back to the Main Document

```javascript.snippet JAVASCRIPT
driver.switch_to.default_content()
```

Switch to the Parent Frame

```javascript.snippet JAVASCRIPT
driver.switch_to.parent_frame()
```

Example

```javascript.snippet JAVASCRIPT
driver.switch_to.frame("my-frame")driver.find_element(    By.ID, "inside-button").click
()
driver.switch_to.default_content()
```

17. Handling Multiple Windows and Tabs

Get Current Window Handle

```javascript.snippet JAVASCRIPT
print(driver.current_window_handle)
```

Get All Window Handles

```javascript.snippet
print(driver.window_handles)
```

Switch to Another Window

```javascript.snippet
original_window = driver.current_window_handledriver.find_element(By.ID, "open-window"
).
click()wait = WebDriverWait(driver, 10)wait.until(    lambda d: len(d.window_handles) >

1)for handle in driver.window_handles:    if handle != original_window:        driver.
switch_to
.window(handle)        break
```

Close the Current Window and Return

```javascript.snippet
driver.close()driver.switch_to.window(original_window)
```

Only close a window if it is appropriate to do so. The original window must still exist before switching back to it.

18. Navigation Methods

```javascript.snippet
driver.get("https://example.com")
```

Open a URL.

```javascript.snippet
driver.back()
```

Go to the previous page.

```javascript.snippet
driver.forward()
```

Go to the next page in browser history.

```javascript.snippet
driver.refresh()
```

Reload the current page.

19. Scrolling a Web Page

Scroll Down

```javascript.snippet
driver.execute_script(    "window.scrollTo(0, document.body.scrollHeight);")
```

Scroll Up

```javascript.snippet
driver.execute_script(    "window.scrollTo(0, 0);")
```

Scroll to an Element

```javascript.snippet
element = driver.find_element(By.ID, "footer")driver.execute_script(
"arguments[0].scrollIntoView({block: 'center'});"
,    element)
```

Scroll by a Fixed Amount

```javascript.snippet
driver.execute_script(    "window.scrollBy(0, 500);")
```

For infinite-scroll pages, scroll and wait for new content to appear. Do not assume that one scroll loads every item.

20. Taking Screenshots

Screenshot of the Current Page

```javascript.snippet
driver.save_screenshot("screenshot.png")
```

Screenshot of an Element

```javascript.snippet
element = driver.find_element(By.ID, "profile")element.screenshot("profile.png")
```

The screenshot files are saved relative to the program's current working directory unless an absolute path is provided.

---

21. Executing JavaScript

Use `execute_script()` to execute JavaScript in the current page.

Get the Page Title

```javascript.snippet
title = driver.execute_script(    "return document.title;")print(title)
```

Get the Page URL

```javascript.snippet
url = driver.execute_script(    "return window.location.href;")print(url)
```

Change an Input Value

```javascript.snippet
element = driver.find_element(By.ID, "username")driver.execute_script(
"arguments[0].value = arguments[1];"
,    element,    "test_user")
```

Important: Directly changing a value using JavaScript may bypass normal user interactions and framework event handling. Prefer `send_keys()` for ordinary form entry and use JavaScript only when necessary.

---

22. Handling File Uploads

For a standard HTML file input, send the full path to the file.

HTML:

Python:

```javascript
file_input = driver.find_element(By.ID, "upload")file_input.send_keys(
"/absolute/path/to/file.pdf"
)
```

On Windows, use a valid path such as:

```javascript
file_input.send_keys(    r"C:\Users\User\Documents\file.pdf")
```

A custom upload button may require locating its underlying file input.

23. Handling Authentication Forms

Example login automation for a website you own or are authorized to test:

```javascript
from selenium import webdriverfrom selenium.webdriver.common.by import Byfrom selenium.
webdriver
.support.ui import WebDriverWaitfrom selenium.webdriver.support import
expected_conditions
 as ECdriver = webdriver.Chrome()wait = WebDriverWait(driver, 10)try:    driver.get("
https
://example.com/login
")    username = wait.until(        EC.visibility_of_element_located(            (By.NAME, "
username")        )    )    password = driver.find_element(        By.NAME, "password
"    )    username.send_keys("
test_user")    password.send_keys("test_password
")    driver.find_element(        By.CSS_SELECTOR, "
button[type='submit']"    ).click()finally:    driver.quit()
```

This is an illustrative example. The actual selectors depend on the target website.

Do not hardcode real credentials in source code. Use environment variables or a secure

secrets manager for sensitive values.

24. Headless Browser

A headless browser runs without displaying a graphical browser window.

Chrome Headless Example

```javascript
javascript.snippet JAVASCRIPT

from selenium import webdriveroptions = webdriver.ChromeOptions()options.add_argument
("--
headless")driver = webdriver.Chrome(options=options)try:    driver.get("https://example
.
com")    print(driver.title)finally:    driver.quit()
```

Useful for:

- Automated test pipelines.
- Continuous integration.
- Background automation.
- Environments without a desktop display.

Headless and headed browsers can behave differently in some environments, so test both when necessary.

25. Browser Options

Configure the browser using options.

```javascript
javascript.snippet JAVASCRIPT

from selenium import webdriveroptions = webdriver.ChromeOptions()options.add_argument
("--
start-maximized")driver = webdriver.Chrome(options=options)
```

Common Chrome Options

```
javascript.snippet JAVASCRIPT

options.add_argument("--headless")options.add_argument("--start-maximized
")options.add_argument("
--incognito")
```

Options and behavior can vary by browser and browser version.

---

26. ActionChains

`ActionChains` supports advanced user interactions, including hovering, dragging, and keyboard or mouse combinations.

Import:

```
javascript.snippet JAVASCRIPT

from selenium.webdriver.common.action_chains import ActionChains
```

Mouse Hover

```
javascript.snippet JAVASCRIPT

menu = driver.find_element(By.ID, "menu")actions = ActionChains(driver)actions.
move_to_element
(menu).perform()
```

Double Click

```
javascript.snippet JAVASCRIPT

element = driver.find_element(By.ID, "button")ActionChains(driver).double_click(element
).
perform()
```

Right Click

```
javascript.snippet JAVASCRIPT

ActionChains(driver).context_click(element).perform()
```

Drag and Drop

```javascript.snippet JAVASCRIPT
source = driver.find_element(By.ID, "source")target = driver.find_element(By.ID,
"target"
)ActionChains(driver).drag_and_drop(    source, target).perform()
```

Click and Hold

```javascript.snippet JAVASCRIPT
ActionChains(driver).click_and_hold(element).perform()
```

Release

```javascript.snippet JAVASCRIPT
ActionChains(driver).release().perform()
```

---

27. Working with Tables

Suppose a page contains an HTML table.

```javascript.snippet JAVASCRIPT
            Name        Age                 Rahim        22                 Karim
23
```

Extract the rows:

```javascript.snippet JAVASCRIPT
table = driver.find_element(By.ID, "students")rows = table.find_elements(By.TAG_NAME,
"tr"
)for row in rows:    cells = row.find_elements(By.CSS_SELECTOR, "th, td")    values = [
cell
.text for cell in cells]    print(values)
```

Example output:

```javascript.snippet JAVASCRIPT
['Name', 'Age']['Rahim', '22']['Karim', '23']
```

---

28. Extracting Links

```javascript.snippet JAVASCRIPT
links = driver.find_elements(By.TAG_NAME, "a")for link in links:    text = link.text

href = link.get_attribute("href")    print(text, href)
```

Some links may have empty text or no `href` attribute, so handle missing values when necessary.

29. Handling Dynamic Content

Dynamic content may load after the initial HTML response.

Use explicit waits:

```javascript.snippet JAVASCRIPT
wait = WebDriverWait(driver, 10)items = wait.until(    EC.
visibility_of_all_elements_located
(        (By.CSS_SELECTOR, ".product")    ))for item in items:    print(item.text)
```

For an element that appears after an action:

```javascript.snippet JAVASCRIPT
driver.find_element(By.ID, "load-more").click()wait.until(    EC.
visibility_of_element_located
(        (By.ID, "new-content")    ))
```

Choose a condition that matches the actual behavior. If content is added while an existing element stays visible, wait for a count increase, new text, or another observable change.

30. Selenium Exceptions

Common exceptions include:

Exception	Meaning
NoSuchElementException	An element could not be found
TimeoutException	A wait condition did not become true in time
ElementNotInteractableException	The element could not be interacted with as attempted
ElementClickInterceptedException	Another element blocked the click
StaleElementReferenceException	The previously located element is no longer attached to the current DOM
InvalidSelectorException	The locator syntax is invalid
NoSuchWindowException	The requested browser window does not exist
NoSuchFrameException	The requested frame could not be found
UnexpectedAlertPresentException	An unexpected JavaScript alert is open
SessionNotCreatedException	The browser session could not be created

Handle a Timeout

```javascript.snippet JAVASCRIPT
from selenium.common.exceptions import TimeoutExceptiontry:    element = WebDriverWait(
driver
, 10).until(        EC.visibility_of_element_located(            (By.ID, "result")

)    )    print(element.text)except TimeoutException:    print("The element did not
become
 visible in time.")
```

Handle a Missing Element

```javascript.snippet JAVASCRIPT
from selenium.common.exceptions import NoSuchElementExceptiontry:    element = driver.
find_element
(By.ID, "optional-element")    print(element.text)except NoSuchElementException:
print
("Element not found.")
```

Handle only exceptions you can meaningfully recover from. Avoid hiding every error with a broad `except Exception` unless you have a clear reason.

31. Stale Element Reference

A stale element error occurs when Selenium holds a reference to an element that is no longer valid in the current DOM.

This can happen when:

- A page reloads.
- A component re-renders.
- A list updates.
- An element is removed and recreated.

Solution

Locate the element again after the DOM changes.

```javascript
javascript.snippet JAVASCRIPT
button = driver.find_element(By.ID, "refresh")button.click()wait = WebDriverWait(driver
,
 10)button = wait.until(    EC.element_to_be_clickable(        (By.ID, "refresh")    ))
```

Do not assume that an old element reference remains valid after navigation or a major page update.

32. Selenium vs Beautiful Soup vs Requests

Feature	Selenium	Beautiful Soup	Requests
Controls a browser	Yes	No	No
Parses HTML	Can access page DOM	Yes	No
Sends HTTP requests	Through browser	No	Yes
Handles JavaScript rendering	Yes, through a browser	No	No
Clicks buttons	Yes	No	No
Fills forms interactively	Yes	No	No

Best use	Browser automation and UI testing	HTML parsing	HTTP requests

Common Combination

```javascript
from selenium import webdriverfrom bs4 import BeautifulSoupdriver = webdriver.Chrome()
try
:    driver.get("https://example.com
")    html = driver.page_source    soup = BeautifulSoup(html, "
html.parser")    print(soup.title.get_text(strip=True))finally:    driver.quit()
```

Selenium loads the page, while Beautiful Soup parses the HTML returned by the browser.

---

33. Complete Example: Search Automation

This example demonstrates a browser search workflow using a public demonstration page. Website selectors and behavior may change.

```javascript
from selenium import webdriverfrom selenium.webdriver.common.by import Byfrom selenium.
webdriver
.common.keys import Keysfrom selenium.webdriver.support.ui import WebDriverWaitfrom
selenium
.webdriver.support import expected_conditions as ECdriver = webdriver.Chrome()wait =
WebDriverWait
(driver, 10)try:    driver.get("https://www.python.org
")    search_box = wait.until(        EC.visibility_of_element_located(            (By.NAME, "
q")        )    )    search_box.send_keys("Python tutorial
")    search_box.send_keys(Keys.ENTER)    wait.until(        EC.url_contains("
search")    )    print("Title:", driver.title)    print("URL:", driver.current_url)
finally
:    driver.quit()
```

The final URL condition is site-dependent; if the site changes its search URL, replace it with a condition appropriate to the actual result page.

---

34. Best Practices

1. Prefer explicit waits over arbitrary sleep calls.
2. Use stable locators such as unique IDs or reliable CSS selectors.

---

3. Use `find_element()` when one element is expected.
4. Use `find_elements()` when zero or more elements may exist.
5. Use `driver.quit()` to clean up the browser session.
6. Avoid mixing implicit and explicit waits.
7. Re-locate elements after DOM updates.
8. Keep test data separate from test logic.
9. Use environment variables or secret storage for credentials.
10. Keep browser and Selenium versions compatible.
11. Avoid unnecessary browser launches.
12. Use headless mode where appropriate.
13. Keep selectors maintainable.
14. Respect website terms, access controls, rate limits, and privacy.
15. Run automated tests against staging or test environments whenever possible.

---

35. Common Mistakes

Mistake 1: Forgetting the Locator Strategy

Incorrect:

```javascript
driver.find_element("username")
```

Correct:

```javascript
driver.find_element(By.ID, "username")
```

Mistake 2: Clicking Before an Element Is Ready

Risky:

```javascript
driver.find_element(By.ID, "submit").click()
```

More reliable:

```javascript.snippet JAVASCRIPT
button = WebDriverWait(driver, 10).until(    EC.element_to_be_clickable(        (By.ID,

"submit")    ))button.click()
```

Mistake 3: Forgetting to Quit the Browser

Use:

```javascript.snippet JAVASCRIPT
driver.quit()
```

Mistake 4: Using `Select` on a Custom Dropdown

`Select` only works with a real HTML element.

Mistake 5: Reusing an Element After Navigation

Locate the element again after navigation or DOM replacement.

---

36. Selenium Project Structure

A small automation project might use this structure:

```javascript.snippet JAVASCRIPT
selenium_project/ main.py requirements.txt pages/    login_page.py    home_page.py
tests
/    test_login.py screenshots/
```

requirements.txt

```javascript.snippet JAVASCRIPT
seleniumpytest
```

Install dependencies:

```javascript.snippet JAVASCRIPT
pip install -r requirements.txt
```

For larger projects, keep browser setup, page objects, test data, and test cases organized in

separate modules.

37. Selenium with Pytest

Pytest is a Python testing framework often used with Selenium.

Install:

```javascript.snippet JAVASCRIPT
pip install pytest selenium
```

Example file: `test_homepage.py`

```javascript.snippet JAVASCRIPT
from selenium import webdriverfrom selenium.webdriver.support.ui import
WebDriverWaitdef
 test_homepage_title():    driver = webdriver.Chrome()    try:        driver.get("https
://
example.com")        assert driver.title == "Example Domain"    finally:        driver.
quit
()
```

Run the test:

```javascript.snippet JAVASCRIPT
pytest -v
```

This test verifies that the page title matches the expected value.

38. Page Object Model (POM)

The **Page Object Model** is a design pattern that separates page locators and interactions from test logic.

Example:

```javascript.snippet JAVASCRIPT
from selenium.webdriver.common.by import Byclass LoginPage:    def __init__(self, driver
):        self.driver = driver    username_input = (By.ID, "username")
password_input
 = (By.ID, "password")    login_button = (By.ID, "login")    def login(self, username, password
):        self.driver.find_element(            *self.username_input        ).send_keys(
username
)        self.driver.find_element(            *self.password_input        ).send_keys(
password
)        self.driver.find_element(            *self.login_button        ).click()
```

Usage:

```javascript.snippet JAVASCRIPT
login_page = LoginPage(driver)login_page.login(    "test_user",    "test_password")
```

The selectors are examples and must match the actual website.

Benefits:

- Less duplicated code.
- Easier maintenance.
- Clear separation of responsibilities.
- Reusable page interactions.

---

39. Selenium 4 Important Features

Selenium 4 introduced or expanded several capabilities, including:

- W3C WebDriver standard compliance.
- Relative locators.
- Improved browser-specific DevTools integration.
- New Window and Tab APIs.
- Selenium Grid improvements.
- Modern Selenium Manager integration in current releases.

Open a New Tab

```
javascript.snippet                                                    JAVASCRIPT

driver.switch_to.new_window("tab")driver.get("https://example.com")
```

Open a New Window

```
javascript.snippet                                                    JAVASCRIPT

driver.switch_to.new_window("window")driver.get("https://example.com")
```

Relative Locator Example

```
javascript.snippet                                                    JAVASCRIPT

from selenium.webdriver.support.relative_locator import locate_withpassword = driver.
find_element
(    By.ID, "password")username = driver.find_element(    locate_with(By.TAG_NAME,
"input"
).above(password))
```

Relative locators can be useful when elements are easier to identify by their relationship to other elements.

---

40. Frequently Asked Interview Questions

Q1. What is Selenium?

Selenium is an open-source browser automation tool used for web application testing and browser interaction.

Q2. What is WebDriver?

WebDriver is the Selenium interface used to control browsers through browser automation commands.

Q3. What is the difference between `find_element()` and `find_elements()`?

`find_element()` returns one matching element and raises an exception if none exists. `find_elements()` returns a list and returns an empty list when there are no matches.

Q4. What is the difference between implicit and explicit waits?

Implicit waits apply to element searches. Explicit waits wait for a specific condition to become

---

true.

Q5. What is XPath?

XPath is a query language for selecting nodes in an XML or HTML document structure.

Q6. What is the difference between CSS selectors and XPath?

CSS selectors use CSS-style patterns to locate elements. XPath can also navigate relationships in the document tree and locate elements using text and other conditions.

Q7. What is `ActionChains`?

`ActionChains` performs advanced interactions such as hovering, dragging, and double-clicking.

Q8. What is an iframe?

An iframe embeds another HTML document inside a page. Selenium must switch to the frame to interact with its contents.

Q9. What is a stale element reference?

It means the previously located element is no longer attached to the current DOM or its reference is no longer valid.

Q10. What is headless mode?

Headless mode runs a browser without displaying its graphical interface.

Q11. What is Selenium Grid?

Selenium Grid distributes automated tests across different machines, browsers, and environments.

Q12. Can Selenium handle JavaScript?

Yes. Selenium controls a real browser, so it can interact with pages after JavaScript executes.

Q13. Can Selenium automate desktop applications?

Selenium is designed for browser automation, not general desktop application automation.

Q14. What is Page Object Model?

POM is a design pattern that organizes page locators and page interactions into reusable classes.

Q15. What is the difference between `close()` and `quit()`?

`close()` closes the current browser window. `quit()` ends the WebDriver session and closes its associated windows.

41. Practice Exercises

Start with these exercises:

1. Open a website and print its title.
2. Print the current URL.
3. Find an element using ID.
4. Find all paragraphs on a page.
5. Enter text into a search box.
6. Click a button after an explicit wait.
7. Select an option from a standard dropdown.
8. Check and uncheck a checkbox.
9. Handle a JavaScript alert.
10. Switch to an iframe.
11. Open a new tab and switch to it.
12. Take a screenshot.
13. Extract all links from a page.
14. Extract HTML table rows.
15. Build a login form test.
16. Write a Pytest test for a page title.
17. Build a reusable Page Object Model class.
18. Handle a stale element reference.
19. Automate a multi-step form in a test environment.
20. Run Selenium in headless mode.

Use websites you own or have permission to test, and avoid automating actions that violate a website's rules.

42. Quick Revision Cheat Sheet

```javascript.snippet JAVASCRIPT
# Browserdriver = webdriver.Chrome()driver.get("https://example.com")# Page informationdriver.title
```

Remember to import the required modules before using these snippets.

Conclusion

Selenium is an important Python tool for browser automation and end-to-end web testing. Learn these topics in order:

1. Installation and browser setup.
2. Locators and element interactions.
3. Explicit waits.
4. Dropdowns, alerts, frames, and windows.
5. Scrolling, screenshots, and JavaScript.
6. Dynamic pages and exception handling.
7. Pytest and Page Object Model.
8. Real-world automation projects.

The most important skills are writing reliable locators, using explicit waits correctly, handling dynamic elements, and designing maintainable automated tests.